Benino Dorsey

Professor Monogioudis

CS375011 Intro to Machine Learning

26 July 2026

Lakehouse Design Report

1. Catalog and Storage Separation

DuckLake keeps metadata in a SQL catalog file and row data in Parquet files in object storage. The catalog stays small because it only holds table definitions, snapshot records, and file paths, never actual rows. This means schema lookups and snapshot queries are fast regardless of how much data is in storage.

For consistency, each snapshot is immutable. A reader that opens a connection at version 7 always sees the same files even if a writer adds version 8 while the read is still running. Writers lock the catalog briefly to record a new snapshot, but readers never block each other.

2. Snapshots, Time Travel, Rollback, and Cost

Every change writes new Parquet files and records a new snapshot. Nothing is overwritten. Time travel works by loading a past snapshot from the catalog and reading only the Parquet files that were active at that point. Newer files are ignored.

Rollback is done by querying the table at a past good snapshot and creating a new table from that result, which records a new recovery snapshot. In this project we corrupted gold.coco_training and recovered it using AT (VERSION => 15).

The cost is storage. Old Parquet files are not deleted automatically when a new snapshot is written. ducklake_expire_snapshots and ducklake_cleanup_old_files remove old snapshots and their files once time travel to them is no longer needed.

3. Uniqueness and Quality Without Constraints

DuckLake has no primary keys or constraints, so quality is enforced in the transform queries. For silver.coco_annotations we used DISTINCT ON (bbox_id) and WHERE area > 0. For silver.visdrone_annotations we used DISTINCT ON (clip_uri, frame_id, target_id) and WHERE category != 'ignored'. Category integers from the Hugging Face dataset were mapped to string labels using a CASE statement so gold tables store readable names rather than index numbers.

4. Tracing an INSERT to Bytes

When INSERT INTO raw.coco_annotations runs, DuckDB serializes the new rows into a Parquet file and writes it to RustFS as an S3 object. DuckLake then writes a new snapshot entry to metadata.ducklake recording the timestamp, the change type, and the path of the new file. The snapshot ID increments. Catalog state lives in metadata.ducklake on the container filesystem and row data lives as Parquet objects inside rustfs-data/ on the host, reachable through the S3 API.

5. SQL Catalog Versus Files Alongside Data

File-only formats like Iceberg and Delta Lake store metadata as JSON files in the same bucket as the data, which removes any external dependency and allows horizontal scaling of writes. DuckLake uses a SQL file instead. This makes catalog queries fast and expressive and keeps writes atomic through database locking. What it makes harder is scale. Only one process can write at a time, and if metadata.ducklake is lost the snapshot history is gone even if all Parquet

files are intact. This tradeoff fits single-machine or small-team workflows well but would not scale to large distributed systems.

6. Why Bytes Stay Out of Tables

Storing image or video bytes in table cells forces DuckDB to read the entire blob on every query, even when only metadata is needed. Instead the tables store URIs pointing to objects in RustFS, and a data loader fetches bytes only for the rows a query returned.

The VisDrone fragment index makes this concrete. silver.visdrone_fragments has one row per fragment with clip_uri, fragment_id, start_frame, end_frame, n_objects, and classes. A query like WHERE n_objects > 20 scans only the small index table and returns matching fragment coordinates. The loader then fetches only those frame files from RustFS by building the S3 key from the clip URI and frame range. No full video is read. DuckDB selects and the loader materializes, which is the core idea the architecture is built on.